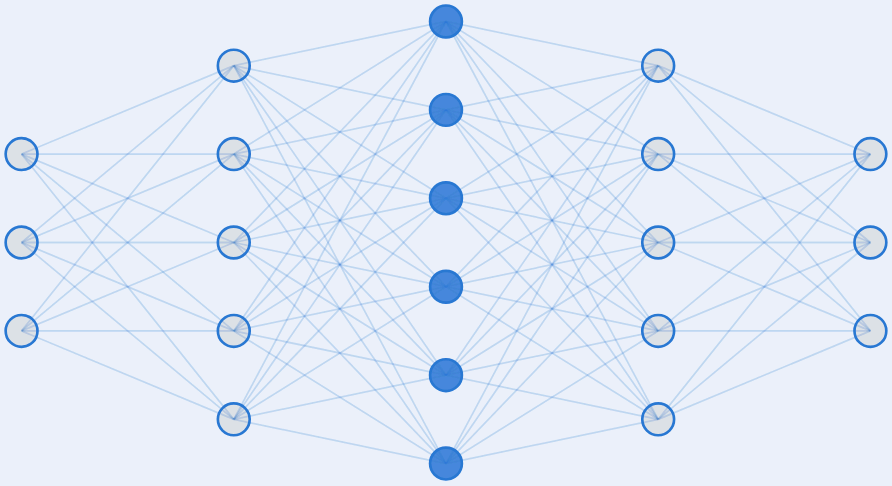
A STEP-BY-STEP GUIDE BOOK

Build Your Own LLM From Scratch

From an empty folder to a small GPT trained on your own data

by Atanu



The cat sat on the ?

predict the next token, again and again

Foundations · Tokenizer · Transformer · Training · Fine-tuning

Chapters 0 to 13, with working PyTorch code

book cover

Build Your Own LLM From Scratch

A step-by-step guide book: from an empty folder to a small GPT that you train on your own data

By Atanu

How to Use This Book

This book teaches you to build a **small but fully working Large Language Model (LLM)** with your own hands, in plain Python and PyTorch. No magic libraries hide the important parts: you will write the tokenizer, the transformer, the training loop and the text generator yourself, and you will understand every line.

What you will build: a GPT-style model of roughly **1 to 85 million parameters** (you choose the size), trained on **your own text** (notes, books, code, articles, chat logs, company documents) and then fine-tuned for a specific purpose.

What you need: basic Python (functions, classes, lists). Everything else, including PyTorch, is explained as we go.

How each chapter works:

1. **Idea**: a plain-language explanation of what the part does and why.
2. **How it works**: the mechanics, with small examples.
3. **Code**: a complete file you can copy, run and modify.
4. **Checkpoint**: a quick test so you know it works before moving on.

By the end you will have these files, built one by one: `clean.py`, `bpe.py`, `prepare.py`, `model.py`, `train.py`, `sample.py` and `sft.py`.

Contents (Index)

Start here: Chapter 0, *Foundations Before You Build*, comes right before Chapter 1. It explains the ideas you need before any code: neural networks, tensors, probability, loss, gradient descent, PyTorch and compute. Read it first; it takes about 30 minutes.

Part	Chapter	What you will learn
I. Foundations	0. Start Here: Foundations Before You Build	The ideas to know before any code
	1. The Big Picture	What an LLM is, and the 7-stage pipeline
	2. Setting Up Your Workshop	Hardware, installation, project layout
II. Data and Tokens	3. Preparing Your Own Data	Collect, clean and size your data
	4. Tokenization	Turn text into numbers with BPE
	5. Building the Training Dataset	Inputs, targets, train and validation split
III. Building the Model	6. Inside the Transformer	Embeddings, attention, MLP
	7. Assembling the Full GPT Model	The complete model.py and model sizes
IV. Training and Generating	8. Training the Model	Loss, optimizer, the training loop
	9. Generating Text	Sampling, temperature, top-k
V. Making It Yours	10. Fine-Tuning on Custom Data	SFT, LoRA and a chat script
	11. Evaluating and Troubleshooting	Tests, checks and fixes
VI. Going Further	12. Scaling Up and Modern Upgrades	RMSNorm, RoPE, GQA, KV cache and more
	13. Resources and Where to Go Next	Books, repositories, papers, a 4-week plan
Appendices	A. Glossary	Key terms in plain words
	B. Command Cheat Sheet and Project Layout	Run order and final folder structure

Part	Chapter	What you will learn
	C. Quick Start on Google Colab	Run the whole book on a free GPU
	D. Frequently Asked Questions	Other languages, PDFs, continuing training

Appendix A: Glossary **Appendix B**: Command cheat sheet and final project layout

0. Start Here: Foundations Before You Build

You can copy code without understanding it, but you cannot debug it, adapt it or trust it. This chapter gives you the ideas that every later chapter assumes. Nothing here needs more than school-level math. Each idea is previewed in plain words with a tiny example, then returns later with real code, and the chapter number tells you where.

If you already know machine learning basics, read section 0.2 and the self-check in 0.13, then jump to Chapter 1.

0.1 The map of ideas

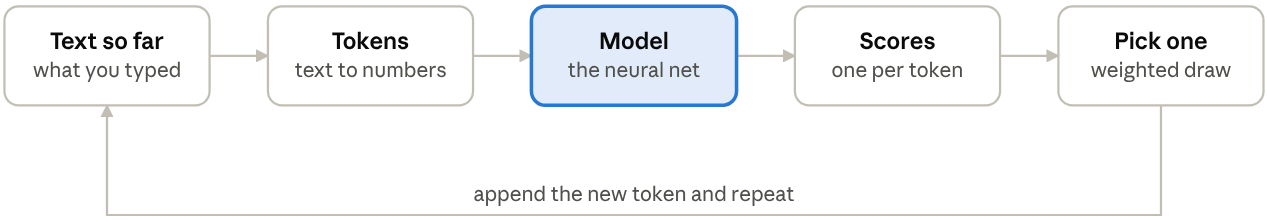
Concept	Plain meaning	Why it matters to a builder	Used in
Neural network	A big function made of simple layers with adjustable numbers	The model you build is one	Ch 6, 7
Parameters (weights)	The adjustable numbers; "11M parameters" means 11 million of them	Model size, memory and quality all follow from this count	Ch 7
Tensor and shape	A grid of numbers with a known shape, like (B, T, C)	Most bugs are shape bugs	Ch 5 to 8
Probability and softmax	Turning scores into chances that add up to 1	How the model chooses the next token	Ch 7, 9
Loss (cross-entropy)	One number saying how wrong the predictions were	The number you watch during training	Ch 8

Concept	Plain meaning	Why it matters to a builder	Used in
Gradient descent and backpropagation	Nudging every parameter downhill on the loss	How learning actually happens	Ch 8
Token and embedding	Text as integers, then integers as meaningful vectors	The model never sees letters, only these	Ch 4, 6
Attention	Tokens looking at earlier tokens to gather context	The core idea of the transformer	Ch 6
Training versus inference	Learning from data versus using what was learned	They need different amounts of compute	Ch 8, 9
Overfitting	Memorizing the training data instead of learning patterns	The main risk when your data is small	Ch 8, 11
Compute and memory	How many numbers must fit and be multiplied	Decides which model size you can train	Ch 2, 7

0.2 What an LLM is, and what it is not

A **large language model** is a program that has learned, from a lot of text, how likely each possible next piece of text is. Give it "The cat sat on the" and it scores every token it knows; "mat" scores high, "carburetor" scores low. It then picks one token, adds it to the text, and repeats.

An LLM writes by repeating one step: predict, pick, append



next-token loop · 5 steps, repeated

What this means in practice:

- **It is not a database or a search engine.** It does not look facts up. Whatever it "knows" is compressed into its parameters as statistical patterns.
- **It is not a rule engine.** Nobody programmed grammar or logic into it. These abilities appear because predicting text well requires them.
- **It optimizes for plausible, not for true.** A fluent but false sentence can score highly. This is called **hallucination**, and it is why you must test model output.
- **It only knows its training data.** The model you build will write like your data and know what your data contains, nothing more. This is the single most important fact for planning a custom model.
- **"Large" is relative.** Frontier models have hundreds of billions of parameters; the models in this book have 1 to 90 million. Same idea, much smaller scale.

0.3 Machine learning in one page

Traditional programming means you write the rules. **Machine learning** means you show examples and let the computer find the rules. Every ML system has three ingredients:

1. **A model:** a function with adjustable numbers (parameters).
2. **A loss:** a score for how wrong the model currently is.
3. **An optimizer:** a procedure that adjusts the parameters to lower the loss.

The basic building block is the **neuron**: multiply each input by a weight, add them up, add a bias, and pass the result through a non-linear **activation function** (such as GELU).

```
output = activation( w1*x1 + w2*x2 + ... + bias )
```

A **layer** is many neurons side by side; a **neural network** is layers stacked. The activation function matters: without it, stacking many layers would collapse into one simple linear function and could not learn language. A network is therefore a very large formula with millions of knobs, and training is the search for good knob settings.

0.4 The only math you need

Three ideas cover almost everything in this book.

- **Vector:** a list of numbers, such as the 384 numbers that represent one token.
- **Dot product:** multiply two vectors element by element and add the results. A large value means the vectors point the same way (they are "similar"). Attention is built on this.

- **Matrix multiplication:** applying a whole grid of weights to a vector (or to many vectors at once). Every layer of the model is a matrix multiplication.

```
import torch

a = torch.tensor([1.0, 2.0, 3.0])                    # a vector, shape (3,)
M = torch.tensor([[1., 0.], [0., 1.], [1., 1.]])     # a matrix, shape (3,
2)

print(torch.dot(a, a))     # 1*1 + 2*2 + 3*3 = tensor(14.)
print(a @ M)               # shape (3,) times (3, 2) gives shape (2,):
tensor([4., 5.])
```

The shape rule: when you multiply `(n, k) @ (k, m)`, the two inner numbers must match and the result is `(n, m)`. When code crashes, nine times out of ten the shapes disagree. Make it a habit to `print(x.shape)` while you build.

0.5 Probability and softmax

The model's final output for each position is a list of raw scores, one per token, called **logits**. Logits can be any number, positive or negative. **Softmax** converts them into a **probability distribution**: every value between 0 and 1, all adding up to exactly 1.

```
import torch

scores = torch.tensor([2.0, 1.0, 0.1])
probs = torch.softmax(scores, dim=0)
print(probs)          # about [0.66, 0.24, 0.10]
print(probs.sum())    # 1.0
```

The biggest score gets the biggest share, but not all of it. To generate text we then **sample** from these probabilities, so the likeliest token is chosen most often but not always. That randomness is why the same prompt can produce different text, and it is what the temperature setting in Chapter 9 controls.

0.6 Loss and cross-entropy

To train, we need one number that says how bad the predictions are. For next-token prediction we use **cross-entropy**: look at the probability `p` the model gave to the token that actually came next, and take the negative natural logarithm.

```
\text{loss} = -\ln p_{\text{correct}}
```

Probability given to the correct token	Loss	Meaning
0.90	0.11	Confident and right
0.50	0.69	Unsure
0.10	2.30	Mostly wrong
0.01	4.61	Very wrong
1 in 2,000 (pure guessing)	7.60	The starting loss of an untrained model with a 2,000-token vocabulary

That last row is the number you will check in Chapter 7. Training is the process of pushing this average loss down.

0.7 How learning works: gradient descent and backpropagation

Imagine the loss as the height of a landscape, where every parameter is a direction you can walk in. You are standing somewhere on a foggy hillside and want to reach the valley. The **gradient** tells you which way is steepest uphill for every parameter at once; you step the opposite way. The size of each step is the **learning rate**: too small and you crawl, too large and you overshoot and bounce around.

Backpropagation is the efficient algorithm that computes the gradient for all millions of parameters in one backward sweep through the network. You never write it yourself: PyTorch's **autograd** records every operation during the forward pass and replays it backwards when you call `loss.backward()`.

Here is the whole idea at toy scale: a model with just two parameters learns the rule `y = 2x + 1` from examples.

```
import torch

x = torch.linspace(-1, 1, 20).unsqueeze(1)    # 20 example inputs
y = 2 * x + 1                                 # the answers we want it to
discover

model = torch.nn.Linear(1, 1)                 # two parameters: a weight and
a bias
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for step in range(200):
    loss = ((model(x) - y) ** 2).mean()       # how wrong are we?
    opt.zero_grad()                           # forget old gradients
    loss.backward()                           # compute new gradients
    opt.step()                                # take a small step downhill

print(model.weight.item(), model.bias.item()) # close to 2.0 and 1.0
```

This is exactly the loop of Chapter 8. A GPT differs only in that the model has millions of parameters instead of two, and the loss is cross-entropy instead of squared error.

0.8 PyTorch survival kit

PyTorch is the library we use for numbers on a CPU or GPU with automatic gradients. You need only this small vocabulary:

You will see	What it does
`torch.tensor(...)`, `torch.randn(...)`	Create a tensor (a grid of numbers)
`x.shape`	The size of each dimension; check it often
`a @ b`	Matrix multiplication
`nn.Linear`, `nn.Embedding`, `nn.LayerNorm`	Ready-made layers that hold parameters
`nn.Module`	The base class for a model: define layers in `__init__`, the computation in `forward`
`loss.backward()`	Compute all gradients (autograd)
`optimizer.step()`	Update parameters using those gradients

You will see	What it does
`model.train()` and `model.eval()`	Switch dropout on (training) or off (generation, evaluation)
`torch.no_grad()`	Skip gradient bookkeeping when you do not need it, which saves memory
`.to(device)`	Move a model or tensor to the GPU or CPU

Autograd in three lines:

```
import torch

x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
y.backward()
print(x.grad)      # tensor(6.), because the slope of x squared is 2x
```

0.9 Language as numbers: tokens, embeddings, context

A computer cannot read letters, so text goes through three conversions that you will build in Chapters 4 and 6:

1. **Tokens:** text is cut into pieces (words, word parts or characters) and each piece gets an integer ID from a fixed list called the **vocabulary**.
2. **Embeddings:** each ID is swapped for a learned vector of numbers. During training, tokens used in similar ways end up with similar vectors, so meaning becomes geometry.
3. **Context window:** the model can only read a fixed number of tokens at once (`block_size`). Anything older is invisible to it. This is its entire working memory.

Word order matters, so the model also adds a **position** signal. Then **attention** lets each token look back at earlier ones and pull in what is relevant, for example connecting "it" to the noun it refers to. That mechanism is the heart of the transformer, and Chapter 6 builds it step by step.

0.10 Pretraining, fine-tuning and inference

Stage	What happens	Data	Compute
Pretraining	Learn language by predicting the next token from scratch	Large raw text	Heavy; most of the cost
Fine-tuning (SFT, LoRA)	Teach a behavior such as answering in a format or persona	Small curated examples	Light; minutes to hours
Inference	Use the trained model to generate text; no learning happens	Your prompt	Light per request

A model straight after pretraining is a **base model**: it continues text but does not follow instructions. A **chat or instruct model** is a base model that was fine-tuned on question and answer examples. You train once and then use the model many times, so training cost is paid up front.

0.11 Practical foundations: overfitting, splits and hardware

Generalization and overfitting. The goal is a model that works on text it has never seen. A model that memorizes its training text scores well there and badly on new text. That is why we always hold back **validation data** that the model never trains on (Chapter 5) and compare the two losses (Chapter 8). The smaller your dataset, the sooner overfitting appears.

Memory. Each parameter is a number that must be stored. Training with the AdamW optimizer in 32-bit precision needs about **16 bytes per parameter**: 4 for the weight, 4 for its gradient, and 8 for the optimizer's two running averages. Activations add more on top. Running a trained model needs only 2 to 4 bytes per parameter.

Model size	Training memory for parameters (about 16 bytes each)	Fits on
1.1 million (Tiny)	about 18 MB	Any laptop
11.5 million (Small)	about 184 MB	Laptop or free GPU

Model size	Training memory for parameters (about 16 bytes each)	Fits on
26 million (Medium)	about 420 MB	Free GPU
87 million (Large)	about 1.4 GB	8 GB GPU with a small batch
1 billion	about 16 GB	A large single GPU
7 billion	about 112 GB	Several GPUs, or use LoRA instead

This table is why this book stays small, and why for big models the practical route is LoRA fine-tuning, not training from scratch.

CPU versus GPU. Training is mostly matrix multiplication, thousands of independent multiply-adds. A GPU does them in parallel, so it is typically many times faster than a CPU. Small models still work on a CPU, just slowly.

Good habits from day one. Fix random seeds when comparing runs, change one setting at a time, and write down the validation loss of every run.

0.12 Building responsibly

- **Data rights:** use text you own or are licensed to use. Public does not always mean free to train on.
- **Privacy:** models memorize. Remove personal details, passwords and keys before training, and never train on other people's private messages.
- **Bias:** a model repeats the patterns, including the unfair ones, in its data. Read samples of your data before you train.
- **Trust:** never present model output as verified fact. State clearly when text was machine-generated.
- **Honest limits:** a small model trained on little data is a learning tool and a style imitator, not an authority.

0.13 Self-check before you continue

Try to answer from memory, then compare.

Question	Short answer
What single task is an LLM trained to do?	Predict the next token given the tokens before it
What are parameters?	The adjustable numbers inside the model that training changes
Why does an untrained model with a 2,000-token vocabulary start at a loss near 7.6?	It guesses uniformly, and the natural log of 2,000 is about 7.6
What does the learning rate control?	How big each parameter update is
Why keep validation data the model never trains on?	To detect overfitting: memorizing instead of learning
Roughly how much memory does training need per parameter?	About 16 bytes with AdamW in 32-bit precision
What is the difference between a base model and a chat model?	A base model continues text; a chat model was fine-tuned on questions and answers

Checkpoint: you can explain, without code, how a model turns text into numbers, scores every possible next token, learns by lowering a loss, and why your data limits what it can know. Ready? Turn to Chapter 1.

1. The Big Picture: What an LLM Really Is

The one idea behind everything

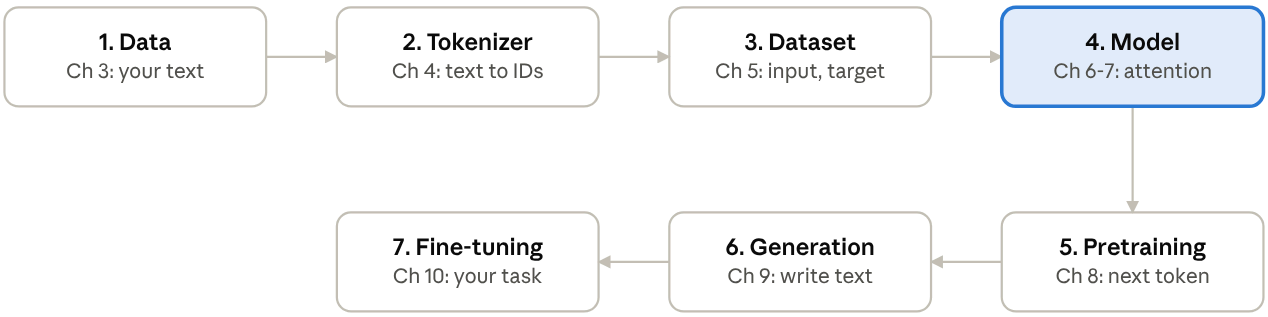
An LLM does exactly one job: **given some text, predict the next piece of text.** That is all. If you type "The capital of France is", the model assigns a high probability to " Paris". To write a whole paragraph, it predicts one token, adds it to the text, and predicts again. Chatting, summarizing, coding and translating are all this same trick, repeated.

Training is how the model learns these probabilities. We show it huge amounts of text, ask "what comes next?" at every position, compare its guess with the real answer, and nudge its internal numbers (the **parameters** or **weights**) so the next guess is slightly better. Repeat millions of times.

The pipeline you will build

Stage	What happens	Chapter
Data	Collect and clean your text	3
Tokenizer	Cut text into tokens, map them to integers	4
Dataset	Make (input, target) training pairs	5
Model	Embeddings, attention, feed-forward layers	6 and 7
Pretraining	Learn language by predicting the next token	8
Generation	Sample new text from the trained model	9
Fine-tuning	Specialize for your task or chat style	10

Seven stages turn raw text into a model that writes



the LLM pipeline · 7 stages

Stages 1 to 4 prepare the data and build the model; stages 5 to 7 train it, use it and customize it. The highlighted model is the part you will understand best by the end.

Reality check: what size can you build?

Frontier models have hundreds of billions of parameters and cost millions of dollars to train. You will not do that on a laptop, and you do not need to. The **architecture is identical** at every size, so a small model teaches you exactly how the big ones work.

Size name	Parameters	Hardware that works	What it can realistically do
Tiny	about 1M	Any laptop CPU	Learn spelling and style of a small text; fun demo
Small	about 11M	Free Colab or Kaggle GPU, or patient CPU	Fluent-looking text in the style of your data
Medium	about 26M	Free GPU, 1 to 2 hours	Better coherence on a focused domain
Large	about 85M	8 to 12 GB consumer GPU	Strong style and domain imitation

Honest expectation: a model trained from scratch on a few megabytes of text will imitate the *style and vocabulary* of that text well, but it will not have general world knowledge or reliable reasoning. For a real assistant, the professional path is to take an open model (Llama, Qwen, Gemma) and fine-tune it, which Chapters 10, 12 and 13 explain. Building from scratch first is the best way to understand what that fine-tuning is actually doing.

Checkpoint: you can explain in one sentence what an LLM does (predict the next token) and name the 7 stages of the pipeline.

2. Setting Up Your Workshop

Hardware options

You have	Use this config (see Chapter 8)	Training time
Laptop, CPU only	Tiny	10 to 30 minutes
Free Google Colab or Kaggle (T4 GPU)	Small	20 to 60 minutes
Gaming GPU (8 GB or more)	Medium or Large	1 to 3 hours
Apple Silicon Mac	Small (uses the `mps` backend)	1 to 2 hours

Install

Use Python 3.10 or newer. Create a clean environment and install only two packages:

```
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install torch numpy
```

Visit pytorch.org for the exact install command if you want GPU support on your machine. Then check what PyTorch can see:

```
import torch
print(torch.__version__)
print("CUDA GPU:", torch.cuda.is_available())
print("Apple GPU:", torch.backends.mps.is_available())
```

Project layout

Create this folder structure. Every script in this book assumes it.

```
my-llm/
├── raw/        # your original .txt files go here
├── data/       # processed data: input.txt, train.bin, val.bin, tok.pkl
├── out/        # saved model checkpoints
├── clean.py  bpe.py  prepare.py
└── model.py  train.py  sample.py  sft.py  chat.py
```

Checkpoint: the Python snippet above runs and prints your PyTorch version.

3. Preparing Your Own Data

Why data matters most

A language model is a compressed copy of its training text. It can only reproduce the style, vocabulary and facts it has seen. **Clean, relevant data beats a large pile of messy data**, especially for small models.

Where to get data

Source	Good for
Your own notes, emails, blog posts, journals	A personal writing-style model

Source	Good for
Source code files from your projects	A code-completion model
Public-domain books (Project Gutenberg)	General language, storytelling
TinyStories dataset	Very clean simple English, ideal for tiny models
Company manuals, FAQs, support tickets	A domain or customer-support model
Wikipedia article dumps	Factual, encyclopedic text

Privacy warning: the model memorizes what it reads. Never train on passwords, API keys, private messages of other people, or anything you would not want it to repeat.

How much data do you need?

A useful guideline from research (the "Chinchilla" rule) is about **20 training tokens per parameter**. For an 11M-parameter model that is about 220 million tokens, close to 1 GB of text. You will often have far less, and that is fine, with these adjustments:

- Under 1 MB of text: use the **Tiny** config, or the model will memorize everything.
- 1 to 20 MB: use **Tiny** or **Small**, keep dropout on, and stop when validation loss stops improving (Chapter 8 shows how).
- 20 MB and up: **Small** or **Medium** works well.

Rough conversion: **1 MB of English text is about 300,000 tokens** with the tokenizer in this book.

Clean and combine your files

Put all your `.txt` files into `raw/`. This script normalizes whitespace and merges everything into one training file.

```
# clean.py
import pathlib
import re

def clean(text):
    text = text.replace("\r\n", "\n")          # unify line endings
    text = re.sub(r"[ \t]+", " ", text)         # collapse spaces and tabs
    text = re.sub(r"\n{3,}", "\n\n", text)      # at most one blank line
    return text.strip()

files = sorted(pathlib.Path("raw").glob("**/*.txt"))
parts = [clean(f.read_text(encoding="utf-8", errors="ignore")) for f in
files]

pathlib.Path("data").mkdir(exist_ok=True)
pathlib.Path("data/input.txt").write_text("\n\n".join(parts), encoding="utf-
8")
print(f"{len(files)} files merged, {sum(len(p) for p in parts):,}
characters")
```

Other cleaning ideas: remove duplicate paragraphs, strip HTML tags, remove boilerplate such as page numbers, and delete very short or garbled lines.

Checkpoint: `data/input.txt` exists and opens as readable text.

4. Tokenization: Turning Text into Numbers

The idea

Neural networks only understand numbers. A **tokenizer** converts text to a list of integers (**token IDs**) and back. The choice of how to split text matters:

Method	Example for "unhappiness"	Pros and cons
Character-level	u, n, h, a, p, p, i, n, e, s, s	Tiny vocabulary, but sequences get very long
Word-level	unhappiness	Short sequences, but unknown words break it and vocabulary is huge

Method	Example for "unhappiness"	Pros and cons
Subword (BPE)	un, happiness	Best balance; used by GPT models

How Byte Pair Encoding (BPE) works

BPE starts with single bytes and repeatedly **merges the most frequent adjacent pair** into a new token.

Take the string `aaabdaaabac`:

1. Most frequent pair is `aa`. Replace it with a new symbol `Z`. Text becomes `ZabdZabac`.
2. Most frequent pair is now `ab`. Replace it with `Y`. Text becomes `ZYdZYac`.
3. Most frequent pair is `ZY`. Replace it with `X`. Text becomes `XdXac`.

The list of merges (in order) **is** the tokenizer. To encode new text, replay the same merges in the same order. Because we start from raw bytes, any text in any language can be encoded, so there are never "unknown" tokens.

Quick-start alternative: character-level

For a first experiment on a tiny file, a character tokenizer is just five lines:

```
text = open("data/input.txt", encoding="utf-8").read()
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}   # char -> id
itos = {i: c for c, i in stoi.items()}       # id -> char
encode = lambda s: [stoi[c] for c in s]
decode = lambda ids: "".join(itos[i] for i in ids)
```

The rest of this book uses BPE, because it makes the model see more text per training step.

Code: a BPE tokenizer from scratch

```
# bpe.py
import re
from collections import Counter

# Splits text into word-like pieces so merges never cross word boundaries
```

```
PAT = re.compile(r"\s?\w+|\s?[^\w\s]+|\s+")

class BPETokenizer:
    def __init__(self):
        self.merges = {}                                  # (id_a, id_b) ->
new id
        self.vocab = {i: bytes([i]) for i in range(256)}  # id -> bytes
        self.cache = {}

    @staticmethod
    def _merge(ids, pair, new_id):
        out, i = [], 0
        while i < len(ids):
            if i < len(ids) - 1 and ids[i] == pair[0] and ids[i + 1] ==
pair[1]:
                out.append(new_id)
                i += 2
            else:
                out.append(ids[i])
                i += 1
        return out

    def train(self, text, vocab_size):
        words = Counter(PAT.findall(text))                      # piece ->
count
        seqs = {w: list(w.encode("utf-8")) for w in words}      # piece ->
byte ids
        for new_id in range(256, vocab_size):
            stats = Counter()
            for w, count in words.items():
                s = seqs[w]
                for pair in zip(s, s[1:]):
                    stats[pair] += count
            if not stats:
                break
            best = max(stats, key=stats.get)                    # most
frequent pair
            for w in seqs:
                seqs[w] = self._merge(seqs[w], best, new_id)
            self.merges[best] = new_id
            self.vocab[new_id] = self.vocab[best[0]] + self.vocab[best[1]]
            if new_id % 200 == 0:
                print(f"merge {new_id - 255}/{vocab_size - 256}")
```

```
    def _encode_piece(self, piece):
        if piece in self.cache:
            return self.cache[piece]
        ids = list(piece.encode("utf-8"))
        while len(ids) >= 2:
            pairs = set(zip(ids, ids[1:]))
            pair = min(pairs, key=lambda p: self.merges.get(p, float("inf")))
            if pair not in self.merges:
                break
            ids = self._merge(ids, pair, self.merges[pair])
        self.cache[piece] = ids
        return ids

    def encode(self, text):
        out = []
        for piece in PAT.findall(text):
            out.extend(self._encode_piece(piece))
        return out

    def decode(self, ids):
        return b"".join(self.vocab[i] for i in ids).decode("utf-8",
errors="replace")
```

Test it

```
from bpe import BPETokenizer

tok = BPETokenizer()
tok.train(open("data/input.txt", encoding="utf-8").read(), vocab_size=1000)
ids = tok.encode("Hello world!")
print(ids)
print(tok.decode(ids))      # must print exactly: Hello world!
```

Choosing the vocabulary size

- **Smaller (500 to 2,000):** fits small datasets, longer sequences, trains faster to a decent result. Recommended here.
- **Larger (30,000 to 100,000):** what big models use; needs far more data, otherwise many tokens are rarely seen.

Checkpoint: `decode(encode(text))` returns exactly the original text.

5. Building the Training Dataset

Next-token prediction: inputs and targets

Suppose your text became the tokens `[5, 9, 2, 7]`. We build two sequences, the second **shifted by one position**:

Position	1	2	3
Input `x`	5	9	2
Target `y`	9	2	7

At position 1 the model sees only `5` and must predict `9`. At position 2 it sees `5, 9` and must predict `2`. One sequence of length T therefore gives **T training examples at once**. The length T is called the **context length** or `block_size`: how far back the model can look.

Train and validation split

We keep 90% of the tokens for **training** and hide 10% as **validation**. The model never learns from validation data, so its loss there tells us honestly whether the model is learning real patterns or just memorizing.

Code: encode everything once

Encoding the full text once and saving it as a binary file makes training fast.

```
# prepare.py
import pickle
import numpy as np
from bpe import BPETokenizer

VOCAB_SIZE = 2000

text = open("data/input.txt", encoding="utf-8").read()
tok = BPETokenizer()
tok.train(text, VOCAB_SIZE)

ids = np.array(tok.encode(text), dtype=np.uint16)
n = int(0.9 * len(ids))
ids[:n].tofile("data/train.bin")
ids[n:].tofile("data/val.bin")

pickle.dump(tok, open("data/tok.pkl", "wb"))
pickle.dump({"vocab_size": len(tok.vocab)}, open("data/meta.pkl", "wb"))
print(f"{len(ids):,} tokens | vocab {len(tok.vocab)} | train {n:,} | val
{len(ids) - n:,}")
```

Run it:

```
python clean.py
python prepare.py
```

The `uint16` type stores each token in 2 bytes, which works for vocabularies up to 65,535.

Checkpoint: `data/train.bin`, `data/val.bin` and `data/tok.pkl` exist, and the printed token count looks reasonable (about 300,000 per MB of text). If it is under about 100,000 tokens in total, use the Tiny config in Chapter 8.

6. Inside the Transformer: Embeddings and Attention

This is the heart of the book. A transformer is built from only a few simple parts, repeated many times. We will meet each part, see what it does with a tiny example, and only then assemble the full model in Chapter 7.

6.1 Notation: the shape of the data

Every tensor inside the model has three dimensions. Learn these letters now; they appear in every code comment.

Letter	Meaning	Example
`B`	Batch size: how many sequences we process at once	32
`T`	Time: number of tokens in each sequence (the context length)	256
`C`	Channels: the size of the vector that represents one token (`n_embd`)	384

So a batch of text, after embedding, is a tensor of shape `(B, T, C)`.

6.2 Embeddings: giving tokens meaning

A token ID such as `1042` is just a label. The model needs a rich vector of numbers it can compute with. An **embedding table** is a matrix with one row per token (`vocab_size` rows, `n_embd` columns). Looking up token 1042 means reading row 1042. Those rows start random and are **learned** during training, so tokens used in similar ways end up with similar vectors.

Attention has no built-in sense of word order, so we also learn a **position embedding**: one vector per position (0, 1, 2, up to `block_size - 1`). The input to the transformer is simply the two added together:

```
x = token_embedding[token_id] + position_embedding[position]
```

6.3 Self-attention: tokens looking at each other

To predict the next word in "The cat sat on the ___", the model must connect "the" with "cat" and "sat". **Self-attention** is the mechanism that lets every token look at the earlier tokens and collect useful information from them.

Each token creates three vectors by multiplying its embedding with three learned matrices:

- **Query (Q):** "what am I looking for?"

- **Key (K):** "what do I contain?"
- **Value (V):** "what information will I hand over if you pick me?"

The steps:

1. Compare the Query of the current token with the Key of every earlier token (a dot product). A high score means "relevant".
2. Divide by the square root of the head size. This keeps scores small so the next step stays stable.
3. Apply **softmax**, which turns the scores into weights that are positive and add up to 1.
4. Output the weighted average of the Values.

As one formula:

```
\mathrm{Attention}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^{\top}}
{\sqrt{d_k}}\right)V
```

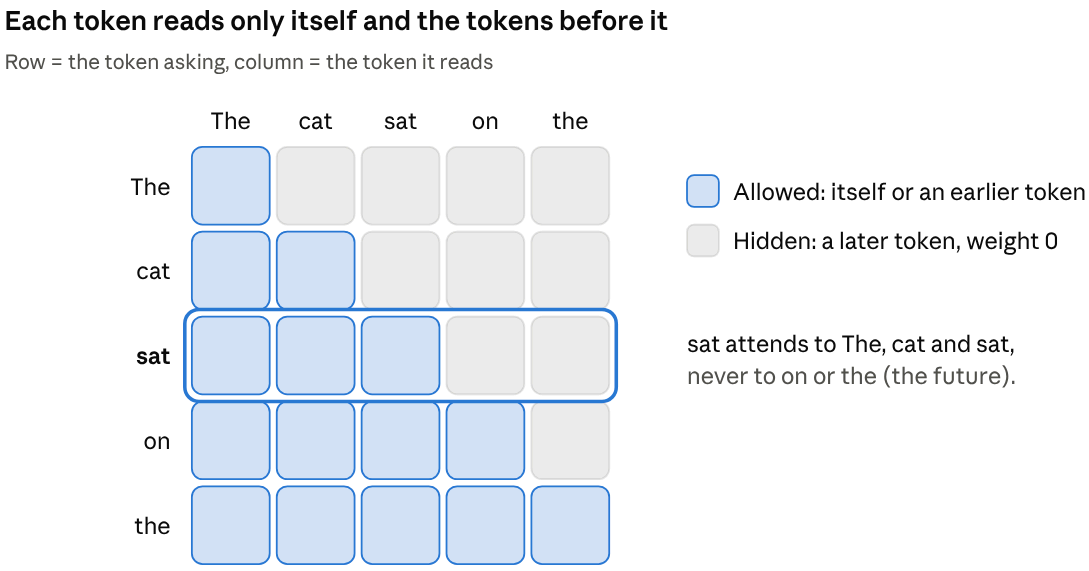
The causal mask. When training, the model sees the whole sentence at once, but it must not peek at future tokens or it could simply copy the answer. We hide the future by setting the scores of all later positions to minus infinity before the softmax, so their weights become exactly 0. This is called **causal** (or masked) attention, and it is what makes the model a text *generator*.

Here it is in 10 lines. Run it and watch the weights: row `t` only has non-zero values up to column `t`.

```
import torch
import torch.nn.functional as F

B, T, C, head = 1, 4, 8, 8
x = torch.randn(B, T, C)
Wq, Wk, Wv = (torch.nn.Linear(C, head, bias=False) for _ in range(3))

q, k, v = Wq(x), Wk(x), Wv(x)
scores = q @ k.transpose(-2, -1) / head ** 0.5       # (B, T, T) how much
each token likes each other token
mask = torch.tril(torch.ones(T, T))                  # lower-triangular
matrix of ones
scores = scores.masked_fill(mask == 0, float("-inf"))   # hide the future
weights = F.softmax(scores, dim=-1)                  # every row sums to 1
out = weights @ v                                    # (B, T, head) weighted
mix of values
print(weights[0])
```



causal attention mask · 5 tokens

The filled lower-left triangle is what each token may read; the empty upper-right triangle is the future, which the mask hides. This is exactly the pattern your `weights` matrix prints.

6.4 Multi-head attention: several views at once

One attention pattern is not enough. One head may track grammar, another may track names, another may look at the previous token. **Multi-head attention** splits the `C` channels into `n_head` groups and runs attention in each group in parallel. With `C = 384` and `n_head = 6`, each head works with 64 channels. The head outputs are concatenated back to size `C` and mixed by one more linear layer (the projection).

6.5 Feed-forward network (MLP): thinking per token

Attention moves information *between* tokens. The **feed-forward network** then processes each token *on its own*: expand the vector 4 times wider, apply a non-linearity (GELU), and shrink back. This is where much of the model's stored knowledge lives, and it holds about two thirds of the parameters.

6.6 Residual connections and LayerNorm: keeping deep networks trainable

- **Residual connection:** each sub-layer adds its result to its own input, `x = x + sublayer(x)`. The original signal always has a clear path through the network, which makes gradients flow and deep stacks trainable.
- **LayerNorm:** rescales each token vector to a stable mean and spread. We apply it *before* each sub-layer (called "pre-norm"), the layout GPT-2 and most modern models use.
- **Dropout:** randomly zeroes a few values during training so the model cannot rely on any single path. Essential for small datasets.

Checkpoint: run the 10-line attention example. The printed 4 by 4 matrix must be lower-triangular (zeros above the diagonal) with each row adding up to 1.

7. Assembling the Full GPT Model

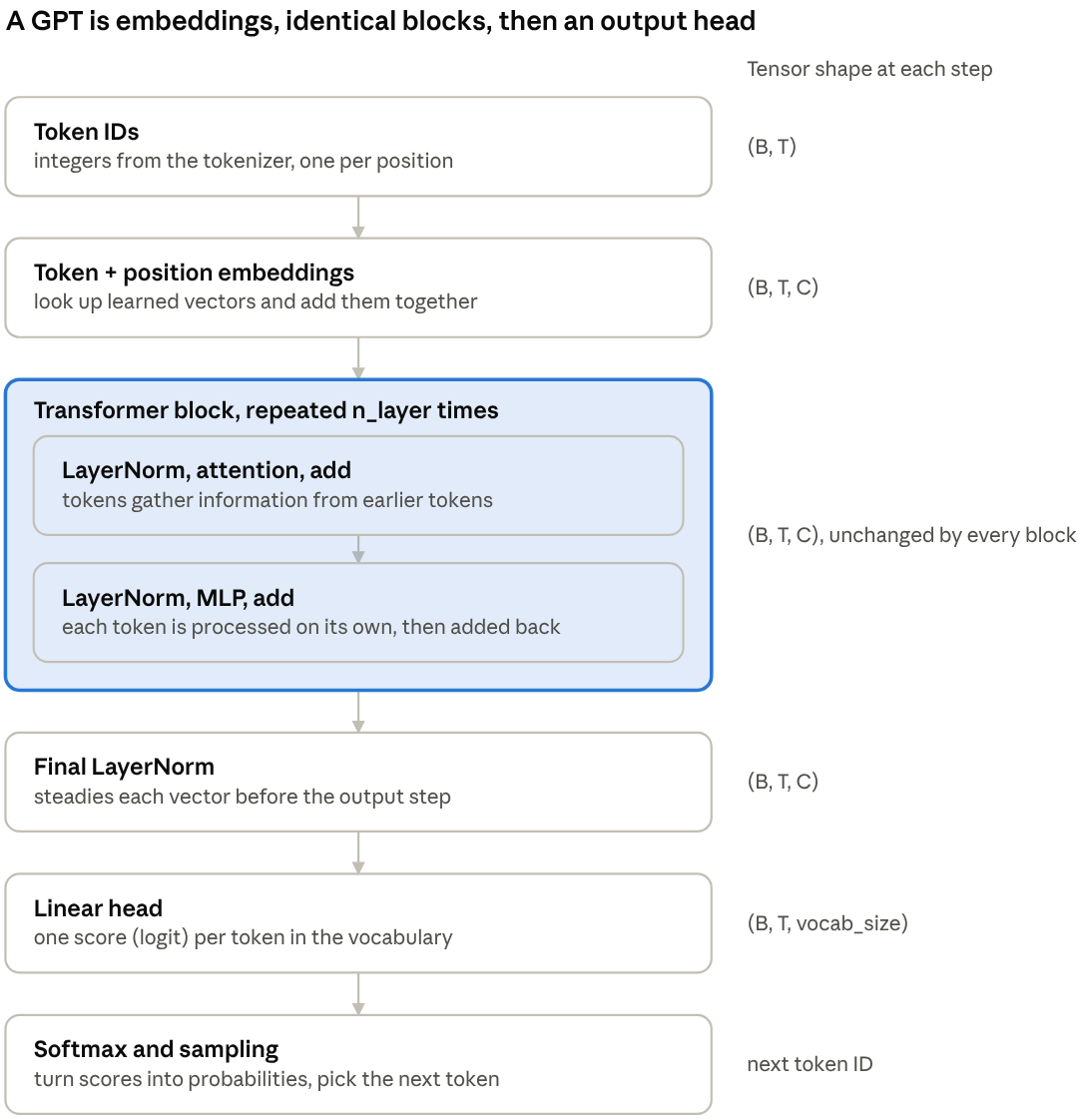
7.1 The recipe

One **transformer block** = LayerNorm, attention, residual add, LayerNorm, MLP, residual add. The full GPT is:

1. Token embedding + position embedding
2. A stack of `n_layer` transformer blocks
3. A final LayerNorm

4. A linear **language-model head** that turns each token vector into `vocab_size` scores (called **logits**), one per possible next token

Two small tricks are used: **weight tying** (the output head reuses the token-embedding matrix, saving parameters and improving quality) and a small-variance **weight initialization** (standard deviation 0.02) for stable early training.



GPT architecture · 6 steps, one repeated block

Data flows top to bottom. The highlighted block is stacked `n_layer` times, and its shape never changes, which is why you can make the model deeper just by raising `n_layer`.

7.2 Code: model.py

```
# model.py
import torch
import torch.nn as nn
from torch.nn import functional as F
from dataclasses import dataclass

@dataclass
class GPTConfig:
    vocab_size: int = 2000
    block_size: int = 256      # context length T
    n_layer: int = 6
    n_head: int = 6
    n_embd: int = 384          # channels C
    dropout: float = 0.1

class CausalSelfAttention(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        assert cfg.n_embd % cfg.n_head == 0
        self.n_head = cfg.n_head
        self.dropout = cfg.dropout
        self.qkv = nn.Linear(cfg.n_embd, 3 * cfg.n_embd, bias=False)   # Q,
K, V in one matrix
        self.proj = nn.Linear(cfg.n_embd, cfg.n_embd, bias=False)
        self.resid_drop = nn.Dropout(cfg.dropout)

    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(C, dim=2)
        # (B, T, C) -> (B, n_head, T, head_size)
        q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
        k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
        v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
        # fused, fast version of: softmax(q k^T / sqrt(d)) with causal mask,
times v
        y = F.scaled_dot_product_attention(
            q, k, v, is_causal=True,
            dropout_p=self.dropout if self.training else 0.0)
        y = y.transpose(1, 2).contiguous().view(B, T, C)               # re-
join the heads
        return self.resid_drop(self.proj(y))
```

```
class MLP(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.fc = nn.Linear(cfg.n_embd, 4 * cfg.n_embd)
        self.proj = nn.Linear(4 * cfg.n_embd, cfg.n_embd)
        self.drop = nn.Dropout(cfg.dropout)

    def forward(self, x):
        return self.drop(self.proj(F.gelu(self.fc(x))))

class Block(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.ln1 = nn.LayerNorm(cfg.n_embd)
        self.attn = CausalSelfAttention(cfg)
        self.ln2 = nn.LayerNorm(cfg.n_embd)
        self.mlp = MLP(cfg)

    def forward(self, x):
        x = x + self.attn(self.ln1(x))     # residual around attention
        x = x + self.mlp(self.ln2(x))      # residual around MLP
        return x

class GPT(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.cfg = cfg
        self.tok_emb = nn.Embedding(cfg.vocab_size, cfg.n_embd)
        self.pos_emb = nn.Embedding(cfg.block_size, cfg.n_embd)
        self.drop = nn.Dropout(cfg.dropout)
        self.blocks = nn.ModuleList([Block(cfg) for _ in range(cfg.n_layer)])
        self.ln_f = nn.LayerNorm(cfg.n_embd)
        self.head = nn.Linear(cfg.n_embd, cfg.vocab_size, bias=False)
        self.head.weight = self.tok_emb.weight          # weight tying
        self.apply(self._init)

    def _init(self, m):
        if isinstance(m, nn.Linear):
            nn.init.normal_(m.weight, mean=0.0, std=0.02)
            if m.bias is not None:
                nn.init.zeros_(m.bias)
        elif isinstance(m, nn.Embedding):
```

```
            nn.init.normal_(m.weight, mean=0.0, std=0.02)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        assert T <= self.cfg.block_size, "sequence longer than block_size"
        pos = torch.arange(T, device=idx.device)
        x = self.drop(self.tok_emb(idx) + self.pos_emb(pos))   # (B, T, C)
        for block in self.blocks:
            x = block(x)
        logits = self.head(self.ln_f(x))                        # (B, T,
vocab_size)
        loss = None
        if targets is not None:
            loss = F.cross_entropy(
                logits.view(-1, logits.size(-1)), targets.view(-1),
                ignore_index=-100)                               # -100 = "do
not train on this"
        return logits, loss

    @torch.no_grad()
    def generate(self, idx, max_new_tokens, temperature=0.8, top_k=50):
        for _ in range(max_new_tokens):
            idx_cond = idx[:, -self.cfg.block_size:]            # crop to the
context window
            logits, _ = self(idx_cond)
            logits = logits[:, -1, :] / max(temperature, 1e-5)  # only the
last position matters
            if top_k:
                v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
                logits[logits < v[:, [-1]]] = float("-inf")
            probs = F.softmax(logits, dim=-1)
            next_id = torch.multinomial(probs, num_samples=1)
            idx = torch.cat([idx, next_id], dim=1)
        return idx
```

7.3 Test the model before training

```
import torch
from model import GPT, GPTConfig

model = GPT(GPTConfig(vocab_size=2000))
print(f"{sum(p.numel() for p in model.parameters()) / 1e6:.1f}M parameters")

x = torch.randint(0, 2000, (2, 16))
logits, loss = model(x, x)
print(logits.shape, loss.item())
```

Expect `torch.Size([2, 16, 2000])` and a loss near **7.6**. That number is not random: an untrained model guesses uniformly among 2,000 tokens, and the natural log of 2,000 is about 7.6. If your first loss is far from `ln(vocab_size)`, something is wrong.

7.4 Choosing your model size

Parameter count is roughly `12 x n_layer x n_embd^2` plus `vocab_size x n_embd`. Pick one row and use it in Chapter 8.

Config	n_layer	n_head	n_embd	block_size	Parameters (vocab 2,000)	Best for
Tiny	4	4	128	128	about 1.1M	CPU, under 1 MB of data
Small	6	6	384	256	about 11.5M	Free GPU, 1 to 20 MB of data
Medium	8	8	512	256	about 26M	Free GPU, 20 MB and up
Large	12	12	768	512	about 87M	8 to 12 GB GPU, 100 MB and up

Checkpoint: the parameter count matches the table and the initial loss is close to 7.6.

8. Training the Model

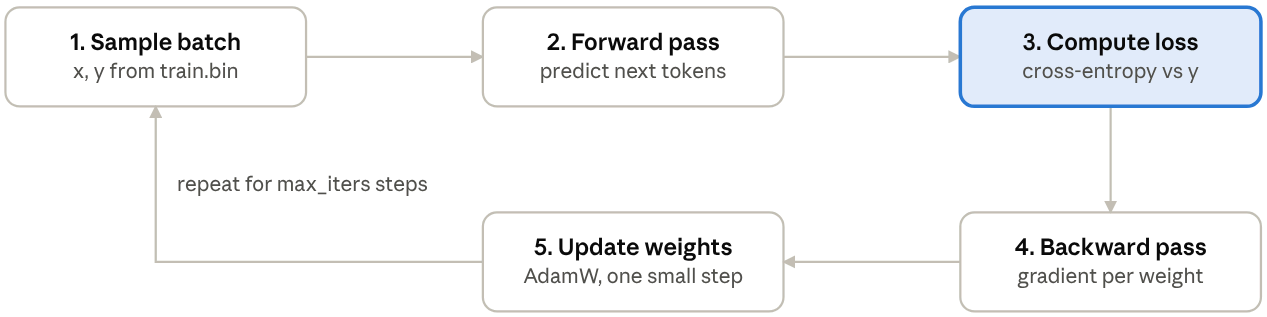
8.1 What training actually does

Training repeats one cycle, thousands of times:

1. **Sample a batch:** pick random chunks of your text, giving inputs `x` and targets `y` (Chapter 5).

2. **Forward pass:** the model predicts a probability for every possible next token at every position.
3. **Loss: cross-entropy** measures how much probability the model gave to the *correct* next token. Giving it 90% costs little; giving it 0.1% costs a lot.
4. **Backward pass:** autograd computes, for every parameter, how a tiny change would change the loss (the **gradient**).
5. **Update:** the optimizer (**AdamW**) moves every parameter a small step in the direction that lowers the loss. The step size is the **learning rate**.

Training is one loop: predict, measure the error, adjust



training loop · 5 steps, repeated

The loss (highlighted) is the number you watch while training; the other four steps exist to make it smaller.

8.2 The knobs that matter

Setting	Meaning	Good starting value
`learning_rate`	Size of each update step	6e-4 for small models
`batch_size`	Sequences per step; bigger is smoother	32 (lower if you run out of memory)
`block_size`	Context length T	256
`max_iters`	Total training steps	3,000 to 10,000
`dropout`	Fraction of values randomly zeroed	0.1 (use 0.2 for very small data)
`weight_decay`	Gentle pull of weights toward zero	0.1

Setting	Meaning	Good starting value
gradient clipping	Caps the update size to avoid blow-ups	1.0

Learning-rate schedule. We start with a short **warmup** (the rate rises from near zero for 200 steps, because early gradients are noisy), then use **cosine decay** down to 10% of the peak. Big steps early, fine steps late.

8.3 Code: train.py

Run it as `python train.py tiny`, `small`, `medium` or `large`, using the presets from Chapter 7.4.

```
# train.py
import math
import os
import pickle
import sys
import time
from contextlib import nullcontext
from dataclasses import asdict

import numpy as np
import torch
from model import GPT, GPTConfig

PRESETS = {
    "tiny":   dict(n_layer=4,  n_head=4,  n_embd=128, block_size=128,
batch_size=64, max_iters=3000),
    "small":  dict(n_layer=6,  n_head=6,  n_embd=384, block_size=256,
batch_size=32, max_iters=5000),
    "medium": dict(n_layer=8,  n_head=8,  n_embd=512, block_size=256,
batch_size=32, max_iters=8000),
    "large":  dict(n_layer=12, n_head=12, n_embd=768, block_size=512,
batch_size=12, max_iters=10000),
}
P = PRESETS[sys.argv[1] if len(sys.argv) > 1 else "small"]

batch_size, block_size, max_iters = P["batch_size"], P["block_size"],
P["max_iters"]
eval_interval, eval_iters = 250, 40
```

```
learning_rate, min_lr, warmup_iters = 6e-4, 6e-5, 200
dropout = 0.1

device = "cuda" if torch.cuda.is_available() else "mps" if
torch.backends.mps.is_available() else "cpu"
use_bf16 = device == "cuda" and torch.cuda.is_bf16_supported()
amp = torch.autocast(device_type="cuda", dtype=torch.bfloat16) if use_bf16
else nullcontext()
torch.manual_seed(1337)
print("device:", device)

# ---------- data ----------
meta = pickle.load(open("data/meta.pkl", "rb"))
data = {s: np.memmap(f"data/{s}.bin", dtype=np.uint16, mode="r") for s in
("train", "val")}
assert len(data["val"]) > block_size + 1, "Not enough data: add more text or
use the tiny preset"

def get_batch(split):
    d = data[split]
    ix = torch.randint(len(d) - block_size - 1, (batch_size,))
    x = torch.stack([torch.from_numpy(d[i:i + block_size].astype(np.int64))
for i in ix])
    y = torch.stack([torch.from_numpy(d[i + 1:i + 1 +
block_size].astype(np.int64)) for i in ix])
    return x.to(device), y.to(device)

# ---------- model ----------
cfg = GPTConfig(vocab_size=meta["vocab_size"], block_size=block_size,
n_layer=P["n_layer"],
                n_head=P["n_head"], n_embd=P["n_embd"], dropout=dropout)
model = GPT(cfg).to(device)
print(f"{sum(p.numel() for p in model.parameters()) / 1e6:.1f}M parameters")

decay = [p for p in model.parameters() if p.dim() >= 2]       # weight
matrices
no_decay = [p for p in model.parameters() if p.dim() < 2]     # biases and
LayerNorm
opt = torch.optim.AdamW(
    [{"params": decay, "weight_decay": 0.1}, {"params": no_decay,
"weight_decay": 0.0}],
    lr=learning_rate, betas=(0.9, 0.95))
```

```
def get_lr(it):
    if it < warmup_iters:
        return learning_rate * (it + 1) / warmup_iters           # linear
warmup
    ratio = (it - warmup_iters) / max(1, max_iters - warmup_iters)
    return min_lr + 0.5 * (1 + math.cos(math.pi * ratio)) * (learning_rate -
min_lr)   # cosine decay

@torch.no_grad()
def estimate_loss():
    model.eval()
    out = {}
    for split in ("train", "val"):
        losses = torch.zeros(eval_iters)
        for k in range(eval_iters):
            x, y = get_batch(split)
            with amp:
                _, loss = model(x, y)
            losses[k] = loss.item()
        out[split] = losses.mean().item()
    model.train()
    return out

# ---------- training loop ----------
os.makedirs("out", exist_ok=True)
best_val, t0 = float("inf"), time.time()
for it in range(max_iters):
    for g in opt.param_groups:
        g["lr"] = get_lr(it)

    if it % eval_interval == 0 or it == max_iters - 1:
        L = estimate_loss()
        print(f"step {it:5d} | train {L['train']:.3f} | val {L['val']:.3f} |
{time.time() - t0:.0f}s")
        if L["val"] < best_val:                                  # keep only
the best model
            best_val = L["val"]
            torch.save({"model": model.state_dict(), "cfg": asdict(cfg)},
"out/ckpt.pt")

    x, y = get_batch("train")
    with amp:
        _, loss = model(x, y)
```

```
    opt.zero_grad(set_to_none=True)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step()

print("done. best val loss:", round(best_val, 3))
```

8.4 Reading the loss curve

The printed `train` and `val` numbers tell you everything.

What you see	What it means	What to do
Starts near 7.6 and drops fast, then slowly	Healthy learning	Keep going
Loss stays near 7.6	Not learning (learning rate or a bug)	Check data, lower the learning rate
`train` keeps falling, `val` rises	**Overfitting**: memorizing, not generalizing	Stop early (the best checkpoint is already saved), more data, higher dropout, smaller model
Loss becomes `nan`	Numerical blow-up	Lower the learning rate, keep gradient clipping
Out-of-memory error	Batch too big for your GPU	Halve `batch_size` or `block_size`

Perplexity is simply `exp(loss)`: roughly how many tokens the model is "confused between" at each step. A loss of 7.6 means confusion between about 2,000 tokens; a loss of 3.0 means about 20. Lower is better, and what is achievable depends heavily on your data.

Checkpoint: `out/ckpt.pt` exists and the validation loss fell well below 7.6.

9. Generating Text

9.1 How generation works

Generation is a loop:

1. Encode your **prompt** into token IDs.
2. Run the model and look only at the scores for the **last** position. These are the logits for the next token.
3. Convert logits to probabilities with softmax and **sample** one token.
4. Append it to the sequence and go back to step 2.

The model never "plans" a sentence. It writes one token at a time, always reading everything before it, and that is enough to produce coherent text.

9.2 Controlling creativity: temperature and top-k

Setting	Effect	Typical range
Temperature	Divides the logits before softmax. Below 1 sharpens the distribution (safe, repetitive); above 1 flattens it (creative, risky).	0.5 to 1.0
Top-k	Keeps only the k most likely tokens and discards the rest, which prevents rare nonsense words.	20 to 50

For factual or domain text use low temperature (0.5 to 0.7). For stories and ideas use 0.9 to 1.0.

9.3 Code: sample.py

```
# sample.py
import pickle
import sys
import torch
from model import GPT, GPTConfig

device = "cuda" if torch.cuda.is_available() else "mps" if
torch.backends.mps.is_available() else "cpu"
ckpt_path = sys.argv[1] if len(sys.argv) > 1 else "out/ckpt.pt"

ckpt = torch.load(ckpt_path, map_location=device)
model = GPT(GPTConfig(**ckpt["cfg"])).to(device)
model.load_state_dict(ckpt["model"])
model.eval()

tok = pickle.load(open("data/tok.pkl", "rb"))
```

```
prompt = "Once upon a time"                          # change this to start of
your own text
idx = torch.tensor([tok.encode(prompt)], device=device)
out = model.generate(idx, max_new_tokens=200, temperature=0.8, top_k=40)
print(tok.decode(out[0].tolist()))
```

```
python sample.py
```

Important: a base model only *continues* text. Give it the beginning of something your data contains (a sentence, a function signature, a heading), not a question. To make it answer questions you need fine-tuning, which is the next chapter.

Checkpoint: the model prints text that matches the style and vocabulary of your data, even if the meaning wanders. That is a successful first LLM.

10. Making It Yours: Fine-Tuning on Custom Data (SFT and LoRA)

10.1 Pretraining versus fine-tuning

	Pretraining (Chapter 8)	Fine-tuning (this chapter)
Goal	Learn language and your domain's patterns	Learn a *behavior*: answer questions, follow a format, use a persona
Data	Large amount of raw text	Small set of examples (hundreds to thousands)
Starts from	Random weights	Your pretrained checkpoint
Cost	Most of the compute	Minutes

10.2 Which customization path should you pick?

Your purpose	Approach	Data you need
Write in my personal style, continue my notes	Pretrain on your text (Chapters 3 to 8)	Your writing, 1 MB and up

Your purpose	Approach	Data you need
Autocomplete code in my project	Pretrain on your code files	Your source files
Answer questions about my product or domain	Pretrain on domain text, then **SFT**	Domain text plus 200 to 2,000 question and answer pairs
Chatbot with a fixed persona or tone	**SFT**	300 to 3,000 example conversations
Adapt a model cheaply, keep the original intact	**LoRA**	Same as SFT

Honest note: SFT on a small from-scratch model teaches it the *format* and *tone* of answers, but it cannot add reasoning or world knowledge the model never saw. For a genuinely capable assistant, apply the exact same SFT or LoRA techniques to a strong open model, as Chapter 13 explains. You will already understand every step.

10.3 Preparing SFT data

Create a file `data/sft.jsonl` with **one JSON object per line**:

```
{"prompt": "What are your opening hours?", "response": "We are open Monday to Saturday, 9 AM to 6 PM."}
{"prompt": "How do I reset my password?", "response": "Open Settings, choose Security, then tap Reset Password."}
```

Every example is turned into one piece of text with fixed markers:

```
### User:
What are your opening hours?
### Assistant:
We are open Monday to Saturday, 9 AM to 6 PM.<|end|>
```

The `<|end|>` marker teaches the model *when to stop talking*. **Quality beats quantity:** 300 clean, consistent examples are better than 3,000 sloppy ones. Keep each example shorter than your `block_size`.

Loss masking. We only want the model to learn the *answer*, not to memorize the questions. So we set the targets of the prompt part to `-100`, which our model's loss function ignores (you already saw `ignore_index=-100` in `model.py`).

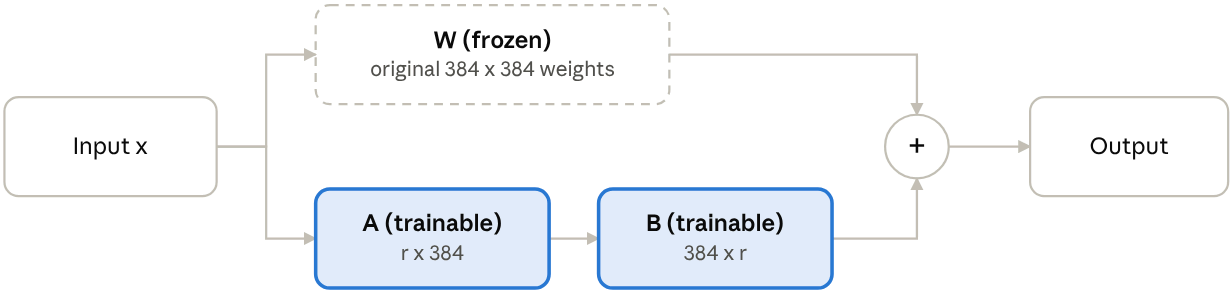
10.4 LoRA: fine-tuning with a tiny number of new parameters

Full fine-tuning updates every weight. **LoRA** (Low-Rank Adaptation) freezes the original weight matrix `W` and learns a small correction as the product of two thin matrices, `B` and `A`:

```
W' = W + \frac{\alpha}{r} \, B A \qquad B \in \mathbb{R}^{d_{out} \times r},\; A \in \mathbb{R}^{r \times d_{in}}
```

For a 384 by 384 layer, the full matrix has 147,456 values; a rank `r = 8` correction has only 6,144 (about 4%). Benefits: far less memory, fast training, and you can keep many small adapters (one per task) for the same base model. The same trick scales to billion-parameter models.

LoRA trains two thin matrices and leaves the original weights frozen



With r = 8, A and B hold 6,144 values instead of 147,456.

LoRA · frozen path plus trainable path

The input flows through the frozen original weights and, in parallel, through the small trainable pair. Their outputs are added, so training changes behavior without touching `W`.

10.5 Code: sft.py (full fine-tuning or LoRA)

Run `python sft.py data/sft.jsonl` for full fine-tuning, or `python sft.py data/sft.jsonl lora` for LoRA.

```
# sft.py
import json
import pickle
import random
import sys
import torch
import torch.nn as nn
```

```
from model import GPT, GPTConfig

device = "cuda" if torch.cuda.is_available() else "mps" if
torch.backends.mps.is_available() else "cpu"
use_lora = "lora" in sys.argv
tok = pickle.load(open("data/tok.pkl", "rb"))

# ---------- LoRA ----------
class LoRALinear(nn.Module):
    def __init__(self, base, r=8, alpha=16):
        super().__init__()
        self.base = base
        for p in self.base.parameters():
            p.requires_grad = False                                    #
freeze original weights
        self.A = nn.Parameter(torch.randn(r, base.in_features) * 0.01)
        self.B = nn.Parameter(torch.zeros(base.out_features, r))     # zeros:
starts as the original model
        self.scale = alpha / r

    def forward(self, x):
        return self.base(x) + (x @ self.A.T @ self.B.T) * self.scale

def add_lora(model, r=8):
    for p in model.parameters():
        p.requires_grad = False
    for blk in model.blocks:
        blk.attn.qkv = LoRALinear(blk.attn.qkv, r)
        blk.attn.proj = LoRALinear(blk.attn.proj, r)

def merge_lora(model):                                                 # fold
adapters back into W
    for blk in model.blocks:
        for name in ("qkv", "proj"):
            layer = getattr(blk.attn, name)
            if isinstance(layer, LoRALinear):
                layer.base.weight.data += (layer.B @ layer.A) * layer.scale
                setattr(blk.attn, name, layer.base)

# ---------- load the pretrained model ----------
ckpt = torch.load("out/ckpt.pt", map_location="cpu")
cfg = GPTConfig(**ckpt["cfg"])
model = GPT(cfg)
```

```
model.load_state_dict(ckpt["model"])
if use_lora:
    add_lora(model)
model.to(device)

# ---------- build training examples ----------
def build(prompt, response):
    p_ids = tok.encode(f"### User:\n{prompt}\n### Assistant:\n")
    r_ids = tok.encode(response + "<|end|>")
    ids = (p_ids + r_ids)[: cfg.block_size + 1]
    labels = ([-100] * len(p_ids) + r_ids)[: cfg.block_size + 1]       # no
loss on the prompt
    return ids, labels

rows = [json.loads(line) for line in open(sys.argv[1], encoding="utf-8") if
line.strip()]
examples = [build(r["prompt"], r["response"]) for r in rows]
print(len(examples), "examples")

def make_batch(exs, bs):
    batch = random.sample(exs, min(bs, len(exs)))
    L = max(len(ids) for ids, _ in batch) - 1
    x = torch.zeros(len(batch), L, dtype=torch.long)
    y = torch.full((len(batch), L), -100, dtype=torch.long)
    for k, (ids, labels) in enumerate(batch):
        n = len(ids) - 1
        x[k, :n] = torch.tensor(ids[:-1])
        y[k, :n] = torch.tensor(labels[1:])
    return x.to(device), y.to(device)

# ---------- train ----------
steps, bs = 300, 8
lr = 1e-3 if use_lora else 1e-4
params = [p for p in model.parameters() if p.requires_grad]
opt = torch.optim.AdamW(params, lr=lr, weight_decay=0.0)
model.train()
for step in range(steps):
    x, y = make_batch(examples, bs)
    _, loss = model(x, y)
    opt.zero_grad(set_to_none=True)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(params, 1.0)
    opt.step()
```

```
    if step % 25 == 0:
        print(f"step {step:4d} | loss {loss.item():.3f}")

if use_lora:
    merge_lora(model)
torch.save({"model": model.state_dict(), "cfg": ckpt["cfg"]}, "out/sft.pt")

# ---------- try it ----------
def ask(question, max_new=150):
    model.eval()
    prompt = f"### User:\n{question}\n### Assistant:\n"
    idx = torch.tensor([tok.encode(prompt)], device=device)
    out = model.generate(idx, max_new, temperature=0.7, top_k=40)
    return tok.decode(out[0].tolist())[len(prompt):].split("<|end|>")
[0].strip()

print(ask(rows[0]["prompt"]))
```

To chat with the result later, load `out/sft.pt` exactly like `sample.py` does and wrap your question in the same `### User:` and `### Assistant:` markers.

10.6 Tips for good fine-tuning

- **Watch for overfitting:** if the model repeats training answers word for word on new questions, lower `steps` or add more varied examples.
- **Be consistent:** the same style, the same format, no contradicting answers.
- **Include variety:** ask the same thing in several different ways.
- **Test on questions you did not train on,** and keep a small list of test prompts to compare before and after.
- **Do not train on private data you would not want repeated,** because the model can reproduce it.

10.7 Chat with your model: chat.py

This small script lets you talk to your fine-tuned model in a loop. It wraps every question in the same `### User:` and `### Assistant:` markers used during fine-tuning, and cuts the answer at `<|end|>`.

```
# chat.py  -  usage: python chat.py out/sft.pt
import pickle
import sys
import torch
from model import GPT, GPTConfig

device = "cuda" if torch.cuda.is_available() else "mps" if
torch.backends.mps.is_available() else "cpu"
ckpt = torch.load(sys.argv[1] if len(sys.argv) > 1 else "out/sft.pt",
map_location=device)
model = GPT(GPTConfig(**ckpt["cfg"])).to(device)
model.load_state_dict(ckpt["model"])
model.eval()
tok = pickle.load(open("data/tok.pkl", "rb"))

print("Ask a question. Press Enter on an empty line to quit.")
while True:
    question = input("\nYou: ").strip()
    if not question:
        break
    prompt = f"### User:\n{question}\n### Assistant:\n"
    idx = torch.tensor([tok.encode(prompt)], device=device)
    out = model.generate(idx, max_new_tokens=150, temperature=0.7, top_k=40)
    text = tok.decode(out[0].tolist())
    print("Model:", text[len(prompt):].split("<|end|>")[0].strip())
```

If the answers are off-topic, that is normal for a small model: add more and more varied examples to `data/sft.jsonl`, or lower the temperature to 0.5.

Checkpoint: after `sft.py`, the model answers a question in your format and stops at `<|end|>` instead of rambling.

11. Evaluating and Troubleshooting

11.1 How to know if your model is good

Numbers and your own eyes both matter.

1. **Validation loss and perplexity.** The lower the validation loss, the better the model predicts text it has never seen. Compare runs only on the same validation data.

2. **A fixed set of test prompts.** Write 10 to 20 prompts that represent your real purpose. After every training run, generate answers and compare them side by side. Human judgement on *your* task is the real test.
3. **A memorization check.** If the model copies long stretches of your training text, it is memorizing rather than learning. Paste a generated passage into this snippet:

```
train_text = open("data/input.txt", encoding="utf-8").read()
generated = "paste the model output here"
for i in range(0, len(generated) - 50, 50):
    chunk = generated[i:i + 50]
    if chunk in train_text:
        print("verbatim copy:", repr(chunk))
```

Many verbatim copies mean overfitting: add data, raise dropout, or use a smaller preset.

11.2 Troubleshooting table

Problem	Likely cause	Fix
Loss stuck near 7.6 (`ln` of vocab size)	Learning rate too low or too high, or a data bug	Print one decoded batch from `get_batch` and read it; try 3e-4
Loss becomes `nan`	Learning rate too high	Halve it; keep gradient clipping on
Output is gibberish	Too few training steps, or the tokenizer in `sample.py` differs from the one used in `prepare.py`	Train longer; always reuse `data/tok.pkl`
Output repeats the same phrase	Too little data, or sampling too greedy	Raise temperature to 0.9, use top-k 40, add data
Val loss much higher than train loss	Overfitting	Stop at the best checkpoint, more data, dropout 0.2, smaller model
`CUDA out of memory`	Batch or context too big	Halve `batch_size` or `block_size`, or use gradient accumulation (Chapter 12)

Problem	Likely cause	Fix
Training very slow on CPU	Expected	Use the `tiny` preset or a free Colab GPU
Error loading a checkpoint (shape mismatch)	The config differs from the one trained	Load the config from the checkpoint, exactly as `sample.py` does
`prepare.py` is slow	Pure-Python BPE on a huge file	Train the tokenizer on a 5 to 10 MB sample, then encode everything

12. Scaling Up and Modern Upgrades

12.1 What really improves a model

Research on scaling laws gives a simple order of priorities for a fixed budget:

1. **More and better data** (the biggest lever for small models).
2. **More training steps** until validation loss stops improving.
3. **A bigger model**, but only if you have the data to feed it (about 20 tokens per parameter).
4. **Architecture tweaks**, which give smaller but real gains.

12.2 Training tricks for limited hardware

Gradient accumulation simulates a large batch on a small GPU: run several small batches, add up the gradients, then take one optimizer step. Replace the training step in `train.py` with:

```
accum = 4                                          # effective batch = batch_size
x 4
opt.zero_grad(set_to_none=True)
for _ in range(accum):
    x, y = get_batch("train")
    with amp:
        _, loss = model(x, y)
    (loss / accum).backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
opt.step()
```

Other options: `torch.compile(model)` for roughly 1.5 times speed on a GPU, and mixed precision (already enabled in `train.py` when your GPU supports bfloat16).

12.3 The modern LLM architecture upgrades

The model you built is GPT-2 style. Modern open models (Llama, Qwen, Gemma and others) keep the same skeleton and swap in these improvements. Each is a small, self-contained change you can try yourself.

Upgrade	What it replaces	Why it helps
RMSNorm	LayerNorm	Simpler and faster, same quality
RoPE (rotary position embeddings)	Learned position table	Encodes relative position by rotating Q and K; generalizes better to longer text
SwiGLU	GELU MLP	A gated feed-forward layer that trains to lower loss
Grouped-Query Attention (GQA)	Full multi-head attention	Shares K and V between heads, so far less memory at generation time
KV cache	Recomputing everything per token	Stores past K and V, making generation much faster
Mixture of Experts (MoE)	One big MLP	Many small MLPs, only a few used per token: huge capacity for modest compute
Flash Attention	Plain attention math	Exact attention with much less memory (already used by `F.scaled_dot_product_attention`)

A first upgrade to try, RMSNorm, is only a few lines. Replace each `nn.LayerNorm(cfg.n_embd)` in `model.py` with it:

```
class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def forward(self, x):
        return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps) *
self.weight
```

12.4 From your model to a real assistant

The professional workflow most people use after learning the internals:

1. Pick a strong open model such as Llama, Qwen or Gemma.
2. Prepare your SFT data exactly as in Chapter 10.
3. Fine-tune with **LoRA or QLoRA** (LoRA on a model compressed to 4-bit) using tools like Unsloth, Axolotl, or Hugging Face TRL with PEFT. A free Colab GPU is often enough for models up to a few billion parameters.
4. Optionally align with preference data using DPO.

You already understand every concept involved, because you built each one.

13. Resources and Where to Go Next

13.1 Books and courses

Resource	What it gives you
Build a Large Language Model (From Scratch) by Sebastian Raschka (Manning, 2024)	The most complete step-by-step path: data handling, embeddings, attention, full transformer, pretraining, classification fine-tuning and instruction fine-tuning

Resource	What it gives you
rasbt/LLMs-from-scratch	The companion repository with chapter notebooks, exercises, solutions, and bonus material such as a Llama-style architecture and LoRA
Andrej Karpathy, "Let's build GPT: from scratch, in code, spelled out" (about 2 hours, video)	A character-level GPT on Shakespeare; the best intuition builder
Andrej Karpathy, "Let's reproduce GPT-2 (124M)" (video)	The full reproduction pipeline at real scale
freeCodeCamp, "LLMs from Scratch: Practical Engineering from Base Model to PPO RLHF" (6+ hours, video)	Pure PyTorch, all the way to supervised fine-tuning, reward modeling and PPO; code at vivekkalyanarangan30/llm_from_scratch

13.2 Code repositories to read and run

Repository	Notes
karpathy/nanoGPT	Minimal, fast GPT training code; the style this book follows
karpathy/build-nanogpt	A commit-by-commit walkthrough of building a GPT-2 reproduction
lerabyte/llm-from-scratch	Seven progressive missions
shreshthtuli/llms-from-scratch	Guided from-scratch implementation
FareedKhan-dev/train-llm-from-scratch	Another end-to-end training example

LitGPT from Lightning AI is also worth exploring once you understand the internals: it lets you load and experiment with many open models.

13.3 Papers worth reading (in this order)

1. *Attention Is All You Need* (the original transformer)
2. The GPT-2 and GPT-3 papers from OpenAI (architecture and scaling insights)

3. *Training Compute-Optimal Large Language Models* (the "Chinchilla" paper, the 20 tokens per parameter rule)
4. *LoRA: Low-Rank Adaptation of Large Language Models*

13.4 Free datasets to practice with

Shakespeare (tiny, for the first test), **TinyStories** (clean simple English, ideal for small models), **Project Gutenberg** public-domain books, **OpenWebText** subsets, and Wikipedia text dumps.

13.5 A suggested learning path

1. **Week 1:** run this book end to end on Shakespeare or TinyStories with the `tiny` or `small` preset.
2. **Week 2:** swap in your own data (Chapter 3), retrain, and compare the generations.
3. **Week 3:** build a small SFT set (Chapter 10) and try both full fine-tuning and LoRA.
4. **Week 4:** add RMSNorm, then RoPE and SwiGLU, and compare validation loss for each change.
5. **After that:** apply LoRA or QLoRA to a strong open model for a production-quality assistant.

Appendix A: Glossary

Term	Plain meaning
Token	A piece of text (a word, part of a word, or a character) turned into an integer
Vocabulary	The complete list of tokens the model knows
BPE	Byte Pair Encoding: build tokens by repeatedly merging the most frequent pair
Embedding	A learned vector of numbers that represents a token or a position
Parameter / weight	A learned number inside the model; "11M parameters" means 11 million of them
Context length (`block_size`)	How many tokens the model can look at once

Term	Plain meaning
Attention	The mechanism that lets each token gather information from earlier tokens
Query, Key, Value	The three vectors each token makes: what it seeks, what it offers, what it hands over
Causal mask	Hides future tokens so the model cannot cheat
Head	One independent attention pattern; multi-head runs several in parallel
MLP / feed-forward	The per-token network that follows attention
Residual connection	Adding a layer's input to its output so signals and gradients flow
LayerNorm	Rescaling each token vector to a stable range
Logits	Raw scores for every possible next token, before softmax
Softmax	Turns scores into probabilities that sum to 1
Cross-entropy loss	How surprised the model is by the true next token; lower is better
Perplexity	`exp(loss)`: the effective number of tokens the model is choosing between
Gradient	The direction in which each parameter should change to reduce the loss
Optimizer (AdamW)	The algorithm that applies the updates
Learning rate	The size of each update step
Epoch	One full pass over the training data
Overfitting	Memorizing the training data instead of learning general patterns
Temperature / top-k	Controls for how random generation is
Pretraining	Learning language by next-token prediction on lots of text

Term	Plain meaning
SFT	Supervised fine-tuning on prompt and response examples
LoRA	Fine-tuning by training small low-rank corrections instead of all weights

Appendix B: Command Cheat Sheet and Final Project Layout

Run order

```
# 1. put your .txt files in raw/
python clean.py                       # merge and clean -> data/input.txt
python prepare.py                     # train tokenizer, encode ->
data/*.bin, data/tok.pkl
python train.py small                 # pretrain -> out/ckpt.pt   (tiny |
small | medium | large)
python sample.py                      # generate text from out/ckpt.pt

# 2. optional: teach a behavior
python sft.py data/sft.jsonl lora     # fine-tune -> out/sft.pt   (omit
'lora' for full fine-tuning)
python chat.py out/sft.pt           # talk to your fine-tuned model
```

Final project layout

```
my-llm/
├── raw/                your original .txt files
├── data/
|   ├── input.txt       merged, cleaned text
|   ├── train.bin       90% of the tokens
|   ├── val.bin         10% of the tokens
|   ├── tok.pkl         the trained tokenizer
|   ├── meta.pkl        vocabulary size
|   └── sft.jsonl       optional fine-tuning examples
├── out/
|   ├── ckpt.pt         best pretrained model
|   └── sft.pt          fine-tuned model
├── clean.py   bpe.py   prepare.py
└── model.py   train.py   sample.py   sft.py   chat.py
```

The whole book on one page

Text, then tokens, then (input, target) pairs, then embeddings, then N blocks of attention and MLP, then logits, then loss, then gradients, then updated weights. Repeat. Then sample. Everything else is detail and scale.

Appendix C: Quick Start on Google Colab (Free GPU)

No GPU on your computer? Colab and Kaggle give you a free one. These steps run the whole book on Colab.

1. Open colab.research.google.com and create a new notebook.
2. Choose **Runtime, then Change runtime type**, and select a **T4 GPU**.
3. Add the book's files: either drag `bpe.py`, `clean.py`, `prepare.py`, `model.py`, `train.py` and `sample.py` into the Files panel on the left, or paste each file into a cell that starts with `%%writefile model.py` (use the right file name each time).
4. Run this cell to create the folders and download a small practice text (Shakespeare, about 1 MB):

```
import os
import urllib.request

os.makedirs("raw", exist_ok=True)
os.makedirs("data", exist_ok=True)
urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/karpathy/char-
rnn/master/data/tinyshakespeare/input.txt",
    "raw/shakespeare.txt")
```

5. Run the pipeline, one command per cell:

```
!python clean.py
!python prepare.py
!python train.py small
!python sample.py
```

6. **Save your work.** Colab erases its files when the session ends. Copy the model to your Google Drive:

```
from google.colab import drive
drive.mount("/content/drive")
!cp out/ckpt.pt data/tok.pkl /content/drive/MyDrive/
```

Notes: a T4 GPU does not support bfloat16, so `train.py` automatically runs in full precision, which is slower but works. With only 1 MB of text, try the `tiny` preset first, because the `small` model may start overfitting early. Kaggle notebooks work the same way: turn on the GPU in the notebook settings and upload your files.

Appendix D: Frequently Asked Questions

Can I train on a language other than English? Yes. The tokenizer works on raw bytes, so any script is supported. Two adjustments help: use a larger `VOCAB_SIZE` in `prepare.py` (4,000 to 8,000), because many scripts use several bytes per character; and for scripts that combine letters with vowel signs (such as Hindi or Bengali), change the splitting pattern at the top of `bpe.py` to `PAT = re.compile(r"\s?\S+|\s+")`, which splits on whitespace only. You still need enough text in that language.

Can I use PDFs or Word files as data? Yes, after converting them to plain text. Extract the text with a library such as `pypdf` (PDF) or `python-docx` (Word), save it as `.txt` files in `raw/`, and run `clean.py`. Scanned PDFs need OCR first.

How do I continue training on new data? Reuse the old tokenizer and the old weights. In `prepare.py`, replace the lines that create and train the tokenizer with `tok = pickle.load(open("data/tok.pkl", "rb"))`. In `train.py`, right after `model = GPT(cfg).to(device)`, add the two lines below and lower `learning_rate` to about 1e-4. Copy `out/ckpt.pt` somewhere safe first, because training overwrites it.

```
start = torch.load("out/ckpt.pt", map_location=device)
model.load_state_dict(start["model"])
```

My model writes nonsense. What now? Check Chapter 11. The usual causes are too little data, too few training steps, or a mismatch between the tokenizer used for training and for sampling.

Can I use the model commercially? The model you train is yours, but the data you train on may carry licenses or privacy limits. Check the rights to your data first (Chapter 0.12), and ask a lawyer for anything beyond personal use.

How do I get a truly capable assistant? Apply LoRA or QLoRA to a strong open model with your own examples (Chapter 12.4). What you learned here carries over directly.